

TP – Contrôleurs (Gestion d’une petite bibliothèque OFPPT)

Nous allons créer une application web simple pour gérer les livres d’une bibliothèque ISTA.

Fonctionnalités demandées :

1. Page d’accueil qui affiche « Bienvenue à la bibliothèque OFPPT »
2. Liste de tous les livres (web)
3. Ajouter / Modifier / Supprimer un livre (web)
4. API REST complète pour les livres (json)

Préparation du projet et de la base de données

1. Créer le projet

```
composer create-project laravel/laravel 007-biblio-ofppt
cd 007-biblio-ofppt
```

Configuration de la base de données

2. # Crée une base MySQL nommée "biblio_ofppt"

On va utiliser SQLite pour simplifier (en cas où la base de données n’existe pas, vous devez créer une en utilisant la commande **touch**) :

```
touch database/database.sqlite
```

3. Fichier .env

```
DB_CONNECTION=sqlite
# DB_CONNECTION=mysql
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=biblio_ofppt
# DB_USERNAME=root
# DB_PASSWORD=
```

1 – Migration + Modèle + Factory + Seeder

Créer le modèle Livre avec migration, factory et resource

```
php artisan make:model Livre -mfsr
```

Migration database/migrations/xxxx_create_livres_table.php

```
public function up(): void
{
    Schema::create('livres', function (Blueprint $table) {
        $table->id();
        $table->string('titre');
```

```

        $table->string('auteur');
        $table->integer('annee');
        $table->string('isbn')->unique();
        $table->timestamps();
    });
}

```

Modèle app/Models/Livre.php

```
protected $fillable = ['titre', 'auteur', 'annee', 'isbn'];
```

Factory database/factories/LivreFactory.php (exemple de 20 livres)

```

class LivreFactory extends Factory
{
    public function definition(): array
    {
        return [
            'titre' => fake()->sentence(3),
            'auteur' => fake()->name(),
            'annee' => fake()->year(),
            'isbn' => fake()->unique()->isbn13(),
        ];
    }
}

```

Seeder database/seeder/LivreSeeder.php

```

// n'oublier pas d'importer le modèle Livre (use App\Models\Livre;)
class LivreSeeder extends Seeder
{
    public function run(): void
    {
        Livre::factory(20)->create();
    }
}

```

Exécuter tout :

```

php artisan migrate
php artisan db:seed LivreSeeder

```

2 – Contrôleur simple (page d'accueil)

```
php artisan make:controller AccueilController
```

app/Http/Controllers/AccueilController.php

```

<?php

namespace App\Http\Controllers;

```

```

use Illuminate\Http\Request;

class AccueilController extends Controller
{
    public function index()
    {
        return view('accueil');
    }
}

```

Vue resources/views/accueil.blade.php

```

<!DOCTYPE html>
<html>

<head>
    <title>Bibliothèque OFPPT</title>
    <meta charset="utf-8">
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>

<body class="bg-gray-100">
    <div class="container mx-auto mt-10">
        <h1 class="text-4xl text-center font-bold text-blue-800">
            Bienvenue à la bibliothèque OFPPT
        </h1>
        <p class="text-center mt-4">
            <a href="{{ route('livres.index') }}" class="text-blue-600 underline">
                Voir tous les livres →
            </a>
        </p>
    </div>
</body>

</html>

```

3 – Contrôleur Resource :

On a déjà créer le contrôleur, si c'est pas le cas, vous pouvez le créer en utilisant la commande suivante :

```
php artisan make:controller LivreController --resource --model=Livre
```

app/Http/Controllers/LivreController.php

```

class LivreController extends Controller
{
    /**
     * Afficher une liste des ressources.
     */
    public function index()
    {
        $livres = Livre::all();
        return view('livres.index', compact('livres'));
    }
}

```

```
}

/**
 * Afficher le formulaire permettant de créer une nouvelle ressource.
 */
public function create()
{
    return view('livres.create');
}

/**
 * Stocker une ressource nouvellement créée dans le stockage.
 */
public function store(Request $request)
{
    $request->validate([
        'titre' => 'required|max:255',
        'auteur' => 'required',
        'annee' => 'required|integer',
        'isbn' => 'required|unique:livres,isbn'
    ]);

    Livre::create($request->all());

    return redirect()->route('livres.index')
        ->with('success', 'Livre ajouté avec succès');
}

/**
 * Afficher la ressource spécifiée.
 */
public function show(Livre $livre)
{
    return view('livres.show', compact('livre'));
}

/**
 * Afficher le formulaire permettant de modifier la ressource spécifiée.
 */
public function edit(Livre $livre)
{
    return view('livres.edit', compact('livre'));
}

/**
 * Mettre à jour la ressource spécifiée dans le stockage.
 */
public function update(Request $request, Livre $livre)
{
    $request->validate([
        'titre' => 'required',
        'auteur' => 'required',
        'annee' => 'required|integer',
        'isbn' => 'required|unique:livres,isbn,' . $livre->id
    ]);
}
```

```

    });

    $livre->update($request->all());

    return redirect()->route('livres.index')
        ->with('success', 'Livre modifié avec succès');
}

/**
 * Supprimer la ressource spécifiée du stockage.
 */
public function destroy(Livre $livre)
{
    $livre->delete();
    return redirect()->route('livres.index')
        ->with('success', 'Livre supprimé avec succès');
}
}

```

4 – Vues Blade (dossier livres)

Crée les fichiers suivants dans **resources/views/livres/** :

layout.blade.php

```

<!DOCTYPE html>
<html>

<head>
    <title>@yield('title') - Bibliothèque OFPPT</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>

<body class="bg-gray-100">
    <nav class="bg-blue-800 p-4">
        <a href="{{ route('accueil') }}" class="text-white font-bold">Accueil</a>
        <a href="{{ route('livres.index') }}" class="text-white ml-6">Livres</a>
    </nav>
    <div class="container mx-auto mt-6">
        @yield('content')
    </div>
</body>

</html>

```

index.blade.php → liste

```

@extends('livres.layout')
@section('title', 'Liste des Livres')
@section('content')
    <h1 class="text-3xl font-bold mb-6 text-center text-blue-800">Liste des Livres</h1>
    <div class="grid grid-cols-1 md:grid-cols-3 gap-6">
        @foreach ($livres as $livre)
            <div class="bg-white p-6 rounded-lg shadow-md">

```

```

        <h2 class="text-xl font-semibold mb-2">{{ $livre->titre }}</h2>
        <p class="text-gray-700 mb-4">Auteur: {{ $livre->auteur }}</p>
        <a href="{{ route('livres.show', $livre->id) }}" class="text-blue-600
hover:underline">Voir
            Détails</a>
    </div>
@endforeach
</div>
@endsection

```

create.blade.php → formulaire ajout

```

@extends('livres.layout')
@section('title', 'Ajouter un Livre')
@section('content')
    <h1 class="text-3xl font-bold mb-6 text-center text-blue-800">Ajouter un Nouveau
Livre</h1>
    <form action="{{ route('livres.store') }}" method="POST" class="max-w-lg mx-auto bg-
white p-6 rounded-lg shadow-md">
        @csrf
        <div class="mb-4">
            <label for="titre" class="block text-gray-700 font-bold mb-2">Titre:</label>
            <input type="text" name="titre" id="titre" class="w-full p-2 border border-
gray-300 rounded" required>
        </div>
        <div class="mb-4">
            <label for="auteur" class="block text-gray-700 font-bold mb-2">Auteur:</label>
            <input type="text" name="auteur" id="auteur" class="w-full p-2 border border-
gray-300 rounded" required>
        </div>
        // Annee
        <div class="mb-4">
            <label for="annee" class="block text-gray-700 font-bold mb-2">Année de
Publication:</label>
            <input type="number" name="annee" id="annee" class="w-full p-2 border border-
gray-300 rounded" required>
        </div>
        //isbn
        <div class="mb-4">
            <label for="isbn" class="block text-gray-700 font-bold mb-2">ISBN:</label>
            <input type="text" name="isbn" id="isbn" class="w-full p-2 border border-gray-
300 rounded" required>
        </div>
        <div class="text-center">
            <button type="submit" class="bg-blue-800 text-white px-4 py-2 rounded
hover:bg-blue-700">Ajouter
                le Livre</button>
        </div>
    </form>
@endsection

```

edit.blade.php → formulaire modification

```

@extends('livres.layout')
@section('title', 'Modifier le Livre')

@section('content')
    <h1 class="text-3xl font-bold mb-6 text-center text-blue-800">Modifier le Livre</h1>
    <form action="{{ route('livres.update', $livre->id) }}" method="POST"
        class="max-w-lg mx-auto bg-white p-6 rounded-lg shadow-md">
        @csrf
        @method('PUT')
        <div class="mb-4">
            <label for="titre" class="block text-gray-700 font-bold mb-2">Titre:</label>
            <input type="text" name="titre" id="titre" value="{{ $livre->titre }}"
                class="w-full p-2 border border-gray-300 rounded" required>
        </div>
        <div class="mb-4">
            <label for="auteur" class="block text-gray-700 font-bold mb-2">Auteur:</label>
            <input type="text" name="auteur" id="auteur" value="{{ $livre->auteur }}"
                class="w-full p-2 border border-gray-300 rounded" required>
        </div>
        <!-- Annee -->
        <div class="mb-4">
            <label for="annee" class="block text-gray-700 font-bold mb-2">Année de
Publication:</label>
            <input type="number" name="annee" id="annee" value="{{ $livre->annee }}"
                class="w-full p-2 border border-gray-300 rounded" required>
        </div>
        <!-- isbn -->
        <div class="mb-4">
            <label for="isbn" class="block text-gray-700 font-bold mb-2">ISBN:</label>
            <input type="text" name="isbn" id="isbn" value="{{ $livre->isbn }}"
                class="w-full p-2 border border-gray-300 rounded" required>
        </div>
        <div class="text-center">
            <button type="submit" class="bg-blue-800 text-white px-4 py-2 rounded
hover:bg-blue-700">Mettre à Jour
                le Livre</button>
        </div>
    </form>
@endsection

```

show.blade.php → détail d'un livre

```

@extends('livres.layout')
@section('title', 'Détails du Livre')
@section('content')
    <h1 class="text-3xl font-bold mb-6 text-center text-blue-800">Détails du Livre</h1>
    <div class="max-w-lg mx-auto bg-white p-6 rounded-lg shadow-md">
        <p class="mb-4"><span class="font-bold">Titre:</span> {{ $livre->titre }}</p>
        <p class="mb-4"><span class="font-bold">Auteur:</span> {{ $livre->auteur }}</p>
        <p class="mb-4"><span class="font-bold">Année de Publication:</span> {{ $livre->annee }}</p>
        <p class="mb-4"><span class="font-bold">ISBN:</span> {{ $livre->isbn }}</p>
        <div class="text-center">

```

```

        <a href="{{ route('livres.index') }}" class="bg-blue-800 text-white px-4 py-2
rounded hover:bg-blue-700">Retour
            à la Liste des Livres</a>
    </div>
</div>
@endsection

```

5 – Contrôleur API Resource

```
php artisan make:controller Api/LivreApiController --api --model=Livre
```

Le contrôleur généré contient déjà :

- index → retourne JSON de tous les livres
- store → crée avec validation
- show → un livre
- update → PUT/PATCH
- destroy → DELETE

```

class LivreApiController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        return Livre::all();
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {
        $request->validate([
            'titre' => 'required|max:255',
            'auteur' => 'required',
            'annee' => 'required|integer',
            'isbn' => 'required|unique:livres,isbn'
        ]);

        $livre = Livre::create($request->all());

        return response()->json($livre, 201);
    }

    /**
     * Display the specified resource.
     */
    public function show(Livre $livre)
    {
        return $livre;
    }
}

```



```

}

/**
 * Update the specified resource in storage.
 */
public function update(Request $request, Livre $livre)
{
    $request->validate([
        'titre' => 'sometimes|required|max:255',
        'auteur' => 'sometimes|required',
        'annee' => 'sometimes|required|integer',
        'isbn' => 'sometimes|required|unique:livres,isbn,' . $livre->id
    ]);

    $livre->update($request->all());

    return response()->json($livre, 200);
}

/**
 * Remove the specified resource from storage.
 */
public function destroy(Livre $livre)
{
    $livre->delete();

    return response()->json(['message' => 'Livre supprimé avec succès'], 200);
}
}

```

6 – Routes

routes/web.php

```

use App\Http\Controllers\AccueilController;
use App\Http\Controllers\LivreController;

Route::get('/', [AccueilController::class, 'index'])->name('accueil');

Route::resource('livres', LivreController::class);

```

routes/api.php

```

<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\LivreApiController;

```

```
Route::post('livres', [LivreApiController::class, 'store']);
Route::put('livres/{livre}', [LivreApiController::class, 'update']);
Route::delete('livres/{livre}', [LivreApiController::class, 'destroy']);
Route::get('livres/{livre}', [LivreApiController::class, 'show']);
Route::get('livres', [LivreApiController::class, 'index']);
```

Si le fichier **api.php** n'existe pas, vous pouvez simplement le créer. N'oubliez pas d'ajouter ce fichier dans le fichier de démarrage **bootstrap/app.php**, ajouter la ligne (api: `__DIR__.'../../routes/api.php'`),)

7 – Test final

Lancer le serveur (garder ouvert dans un terminal)

```
php artisan serve
```

Exécutez le serveur de développement Vite.

```
npm install
npm run dev
```

1. Web
 - <http://127.0.0.1:8000> → Accueil
 - <http://127.0.0.1:8000/livres> → Liste + CRUD complet
2. API (avec Httpie ou curl)

```
GET      http://127.0.0.1:8000/api/livres
POST     http://127.0.0.1:8000/api/livres
        Content-Type: application/json
        {"titre":"Laravel
12","auteur":"OFPPT","annee":2025,"isbn":"9999999999999"}
```

Exercices :

1. Ajoute un champ editeur dans la table livres et mets à jour TOUTES les vues + contrôleurs + API.
2. Dans LivreController@store, redirige vers livres.show après création au lieu de livres.index.
3. Fais en sorte que l'API retourne uniquement id, titre et auteur (modifie la Resource).
4. Affiche le nombre total de livres sur la page d'accueil avec un contrôleur simple.